Práctica de Laboratorio

Construye tu propio
Recomendador de Películas

con Apache Spark y Machine Learning en Databricks

Asignatura: Tecnologías del Big Data
Nivel: Introductorio | Herramienta: Databricks Free Edition
Dataset: MovieLens (100.000 valoraciones reales)

1. ¿De qué va esta práctica?

Seguro que alguna vez has pensado: **"¿cómo sabe Netflix lo que me gusta?"** La respuesta es un **sistema de recomendación**: un programa que analiza lo que has visto (y cómo lo has valorado) para predecir qué otras cosas podrían gustarte.

En esta práctica vamos a construir uno desde cero. Usaremos datos reales de valoraciones de películas, los limpiaremos, entrenaremos un modelo de Machine Learning y al final obtendremos recomendaciones personalizadas para cada usuario.

No te preocupes si nunca has tocado Spark o Machine Learning. Los notebooks están comentados paso a paso y esta guía te explica el porqué de cada decisión.

💡	**Consejo** Los sistemas de recomendación están en todas partes: Netflix, Spotify, Amazon, YouTube... Todos usan variantes del mismo tipo de algoritmo que vamos a implementar aquí.

2. ¿Cómo funciona un recomendador?

Antes de escribir una sola línea de código conviene entender la idea principal. Hay dos grandes familias de recomendadores:

2.1 Basado en contenido

Recomienda cosas **parecidas a lo que ya te ha gustado**. Si te gustó una película de ciencia ficción con naves espaciales, te recomienda otras con naves espaciales.

- Ventaja: **no necesita datos de otros usuarios**

- Desventaja: **nunca te saca de tu zona de confort** (si solo has visto acción, solo te recomienda acción)

2.2 Filtrado colaborativo

Recomienda cosas que **le han gustado a personas con gustos similares a los tuyos**. La lógica es: "si tú y otro usuario habéis valorado igual 20 películas, probablemente las que le gustaron a él también te gusten a ti".

- Ventaja: **descubre cosas que no habrías buscado tu solo**
- Desventaja: **necesita suficientes valoraciones para funcionar** (problema del arranque en frío)

> 📌 **Recuerda**
> En esta práctica usaremos filtrado colaborativo. Es el enfoque más habitual en la industria y el que mejores resultados suele dar cuando hay suficientes datos.

2.3 El algoritmo ALS

El corazón de nuestra implementación es **ALS (Alternating Least Squares)**. Imagínalo así:

Tenemos una tabla enorme donde las filas son usuarios y las columnas son películas. Cada celda debería contener la valoración de ese usuario para esa película. El problema es que la inmensa mayoría de celdas están vacías: ningún usuario ha visto todas las películas.

ALS rellena esa tabla. Lo hace buscando patrones ocultos: representa a cada usuario y cada película como un conjunto de números (sus **"factores latentes"**), y aprende esos números de tal forma que al multiplicarlos reproduzcan las valoraciones reales conocidas. Una vez aprendidos, puede predecir cualquier celda vacía.

> 💡 **Consejo**
> El parámetro clave de ALS es el 'rank': cuántos factores latentes usa. Más rank = modelo más complejo = puede aprender patrones más sutiles, pero también tarda más en entrenar y puede sobreajustarse.

3. Los datos: MovieLens

Trabajaremos con el dataset MovieLens, uno de los benchmarks más usados en investigación de sistemas de recomendación. Contiene valoraciones reales de usuarios de la plataforma MovieLens recogidas a lo largo de varios años.

El dataset se compone de cuatro ficheros CSV que se relacionan entre sí a través del campo movieId:

movies.csv (9.742 películas)

- `movieId` — identificador único de la película
- `title` — título con el año de estreno entre paréntesis: `Toy Story (1995)`
- `genres` — géneros separados por `|`: `Action|Comedy|Drama`

ratings.csv (100.836 valoraciones)

- `userId` — identificador del usuario
- `movieId` — película valorada
- `rating` — puntuación de 0.5 a 5.0 estrellas en incrementos de 0.5
- `timestamp` — fecha de la valoración en formato Unix epoch

> 📌 **Recuerda**
> Este es el fichero más importante. Sin valoraciones no hay recomendaciones. Asegúrate de cargarlo correctamente antes de continuar.

links.csv (9.742 entradas)

- Relaciona cada película con su ID en **IMDb** y **TMDb** para poder enlazar a información externa.

trailers.csv (25.623 entradas)

- Asocia cada película con uno o varios tráileres de YouTube.
- Hay más filas que películas porque una película puede tener varios tráileres.

> ⚠ **Atención**
> Los CSV tienen algunas imperfecciones: el campo title mezcla título y año en una sola columna, el timestamp no es una fecha legible, y el campo genres es una cadena de texto en lugar de una lista. Parte del ejercicio consiste en limpiar todo esto.

4. La arquitectura Lakehouse

Antes de ponernos a programar es importante entender cómo vamos a organizar los datos. En proyectos de Big Data es muy común usar una arquitectura por capas llamada Lakehouse:

Capa Bronze - los datos tal cual llegan

Los datos se guardan **exactamente como vienen de la fuente**, sin ninguna modificación. Son los CSV originales. Si algo falla en capas superiores, siempre podemos volver aquí a empezar de nuevo.

Capa Silver - datos limpios

Aquí se corrigen errores, se ajustan tipos de datos, se eliminan registros inválidos y se estructuran los datos de forma consistente. Los datos Silver son **fiables y listos para analizar**.

Capa Gold - datos listos para consumir

Es el resultado final: tablas optimizadas listas para ser consultadas por aplicaciones, dashboards o usuarios finales. En nuestro caso, la tabla de **recomendaciones por usuario** será nuestra capa Gold.

> 💡 **Consejo**
> Esta separación en capas parece excesiva para un ejercicio pequeño, pero en proyectos reales es fundamental: permite rehacer solo la parte que falla sin tener que reprocesar todo desde el principio.

El flujo completo de la práctica sigue este esquema:

Capa	Fase	Qué hay que hacer
Bronze	Carga de datos	Leer los CSV tal cual llegan y convertirlos en DataFrames de Spark.
Silver	Limpieza	Corregir tipos, separar campos, construir URLs. Dejar los datos listos para analizar.
Silver	Transformación	Entrenar el modelo ALS y generar las recomendaciones por usuario.
Gold	Servicio	Guardar las recomendaciones como tabla Delta para que puedan consultarse.
Gold	Consumo	Mostrar las recomendaciones de forma visual con trailers y enlaces.

5. Qué tienes que hacer

La práctica se divide en cinco notebooks que deben ejecutarse en orden. A continuación se describe el objetivo de cada uno y los puntos clave en los que tienes que prestar atención.

Notebook 00 - Entorno

Configura las credenciales de acceso al almacenamiento en la nube y define las rutas a los datos.

Tu tarea: Rellenar las variables `ADLS_ACCOUNT` y `ADLS_SAS_KEY` con los datos que te facilitará el profesor.

> ⚠ **Atención**
> Este notebook no produce ningún resultado visible por sí solo, pero todos los demás dependen de él. Si las credenciales son incorrectas, fallará todo lo demás.

Notebook 01 - Arquitectura

Muestra un diagrama visual de la arquitectura Lakehouse. Solo es informativo; no hay código que ejecutar.

Notebook 02 - Limpieza

Aquí se cargan los cuatro CSV desde la capa Bronze y se transforman en DataFrames limpios que se guardan en Silver.

Puntos clave:

1. Ejecuta las celdas en orden. Algunas celdas usan variables definidas en celdas anteriores.
2. Observa cómo `printSchema()` muestra los tipos de datos antes y después de la limpieza. Es normal que al principio todos los campos sean de tipo `string`.
3. Fíjate en la función que separa el título del año: usa una expresión regular. No es necesario que la entiendas en detalle, pero sí que sepas qué hace.
4. Al final, los cuatro DataFrames se guardan como tablas **Delta**. Esto es importante: a partir de aquí los datos viven en tablas, no en ficheros.

> ⛔ **Error típico**
> No modifiques el orden de las celdas. Si ejecutas la celda de limpieza antes de cargar los datos, obtendrás un error de variable no definida.

Notebook 03 - transformación y modelo

Este es el núcleo de la práctica. Aquí se entrena el modelo de Machine Learning y se generan las recomendaciones.

Puntos clave:

5. El dataset se divide en **80% para entrenar** y **20% para evaluar**. Esta es una práctica estándar en ML: no puedes evaluar el modelo con los mismos datos que usaste para entrenarlo.
6. Se usa **validación cruzada (Cross Validation)** para encontrar el mejor valor del parámetro `rank`. El proceso prueba varios valores y se queda con el que da menor error. Este paso puede tardar varios minutos.
7. La calidad del modelo se mide con **RMSE (Root Mean Squared Error)**: cuanto menor sea, mejor predice el modelo las valoraciones reales. Un RMSE de 1.0 significa que nos equivocamos en promedio en 1 estrella.

8. El modelo final se reentrena con TODOS los datos (incluyendo un usuario ficticio) antes de generar las recomendaciones.

> 💡 **Consejo**
> El usuario ficticio (userId=0) está precargado con valoraciones de películas clásicas. Cuando ejecutes el notebook, las recomendaciones que reciba deberán ser coherentes con esas preferencias. Es una buena forma de verificar que el modelo funciona.

> 🚫 **Error típico**
> En Databricks Free Edition la función automática de recomendaciones del modelo no funciona correctamente. El notebook incluye un workaround que la calcula manualmente. No te preocupes si ves un bloque de código largo con matrices: hace exactamente lo mismo, solo que de forma manual.

Notebook 04 - Consumo

Muestra las recomendaciones de forma visual: trailers embebidos de YouTube y enlaces a IMDb y TMDb.

Tu tarea: Prueba a cambiar el `user_id` en la última celda para ver las recomendaciones de distintos usuarios. Observa como cambian según el usuario.

6. Errores típicos y cómo evitarlos

Estos son los problemas que más frecuentemente aparecen la primera vez que se trabaja con Spark y con modelos de recomendación:

El cluster no está arrancado

Databricks necesita un **cluster de cómputo** activo para ejecutar código. Si no tienes ninguno conectado, ningún notebook funcionará. **Antes de ejecutar cualquier celda**, verifica en la parte superior que hay un cluster seleccionado y que su estado es `Running`.

Variable no definida (NameError)

Spark y Python son secuenciales: una celda puede usar variables definidas en celdas anteriores, **pero solo si esas celdas se han ejecutado antes**. Si ejecutas las celdas en desorden o reinicias el kernel sin volver a ejecutar desde el principio, aparecen errores del tipo `NameError: name 'movies_raw' is not defined`.

> 💡 **Consejo**
> Si algo falla de forma inexplicable, prueba a usar 'Run all' para ejecutar el notebook entero desde el principio. Muchos errores misteriosos desaparecen solos.

Error de credenciales en el Notebook 00

Si `ADLS_ACCOUNT` o `ADLS_SAS_KEY` son incorrectos, la lectura de cualquier CSV fallará con un error de autorización. Comprueba que has copiado las credenciales exactamente como te las ha dado el profesor, sin espacios extra ni caracteres de más.

Todos los tipos son 'string' despues de leer el CSV

Cuando Spark lee un CSV con `inferSchema=false` (o sin especificarlo), interpreta todas las columnas como texto. Es normal ver `userId: string` en el schema inicial. La limpieza del Notebook 02 se encarga de convertirlos al tipo correcto. Si omites esas conversiones, el modelo de ML fallara porque ALS necesita numeros, no texto.

El entrenamiento tarda mucho

La validación cruzada entrena **varios modelos en paralelo**. En un cluster pequeño (como el de la edición gratuita de Databricks) puede tardar entre 5 y 15 minutos. Es completamente normal. No cierres el notebook ni interrumpas la ejecución.

> ⚠ **Atención**
> Si el entrenamiento supera los 20 minutos sin completarse, es posible que el cluster se haya quedado sin recursos. Consulta al profesor.

El modelo recomienda siempre las mismas películas

Si todas las recomendaciones son idénticas para todos los usuarios, probablemente el modelo no ha aprendido bien. Las causas más comunes son: **muy pocas iteraciones** (`maxIter` demasiado bajo), un **rank demasiado pequeño**, o que el dataset de entrenamiento haya quedado muy desbalanceado en la división 80/20.

Recomendaciones incoherentes para el usuario ficticio

El usuario ficticio (userId=0) tiene valoraciones de 5 estrellas para películas de ciencia ficción y thriller. Si sus recomendaciones son comedias románticas o animación infantil, algo no ha ido bien en el reentrenamiento. Verifica que la celda que une los DataFrames (`unionByName`) se ha ejecutado antes de la de entrenamiento.

7. Conceptos clave que deberías retener

No se espera que salgas de esta práctica siendo un experto en Machine Learning. Pero sí que te quedes con estas ideas:

- **DataFrame de Spark**: una tabla distribuida que puede procesar millones de filas en paralelo sobre múltiples máquinas. Es la estructura de datos fundamental en Big Data.

- **Esquema (schema)**: la estructura de un DataFrame: nombres de columnas y sus tipos. Verificar el schema después de cada transformación es un hábito que te ahorrará muchos errores.

- **Transformaciones lazy**: en Spark, operaciones como `select()` o `filter()` no se ejecutan inmediatamente. Solo se ejecutan cuando se necesitan los datos de verdad (por ejemplo, al llamar a `display()` o `write()`). Esto hace que Spark sea muy eficiente.

- **Formato Delta**: un formato de almacenamiento avanzado que aporta fiabilidad, velocidad y la posibilidad de hacer actualizaciones eficientes. Es el estandar en entornos Databricks.

- **Sobreajuste (overfitting)**: cuando un modelo aprende tan bien los datos de entrenamiento que falla con datos nuevos. La validacion cruzada y el conjunto de test existen precisamente para detectarlo.

- **RMSE**: métrica de error para problemas de regresión. Un RMSE de 0.88 significa que, de media, el modelo se equivoca en 0.88 estrellas al predecir una valoración.

- **Filtro de arranque en frío (cold start)**: el problema de un recomendador cuando aparece un usuario nuevo sin historial, o una película sin valoraciones. ALS lo gestiona con el parámetro `setColdStartStrategy('drop')`, que directamente descarta esas predicciones.

8. Si quieres saber más

La práctica cubre los fundamentos, pero hay mucho más que explorar. Algunas ideas si quieres ir más allá:

- **Prueba diferentes valores de rank**: modifica el grid de búsqueda en el Notebook 03 para incluir valores como 20 o 50. Observa cómo cambia el RMSE y el tiempo de entrenamiento.
- **Añade más películas al usuario ficticio**: cambia las valoraciones del userId=0 por tus películas favoritas y comprueba si las recomendaciones tienen sentido.
- **Explora el dataset con SQL**: la capa Silver ya está guardada como tablas. Puedes hacer consultas directamente: cuáles son las películas mejor valoradas, qué usuarios han valorado más películas, qué géneros son más populares...
- **Documentacion oficial de Spark MLlib**: https://spark.apache.org/docs/latest/ml-collaborative-filtering.html

Checklist de entrega

Antes de dar la práctica por terminada, verifica que:

9. El Notebook 00 se ejecuta sin errores y muestra 'Entorno listo'.
10. El Notebook 02 completa la limpieza y las cuatro tablas Silver existen en el catálogo de Databricks.
11. El Notebook 03 muestra un RMSE final inferior a **1.0**.

12. La tabla workspace.recommender.user_recommendations contiene recomendaciones para al menos 50 usuarios distintos.
13. El Notebook 04 muestra los trailers y enlaces de las recomendaciones del usuario que elijas.



Recuerda
Si todos los puntos anteriores se cumplen, la práctica está completada correctamente. El objetivo no es obtener el RMSE más bajo del mundo, sino entender el flujo completo de un proyecto de Big Data y Machine Learning.